

Before you turn this problem in, make sure everything runs as expected. First, **restart the kernel** (in the menubar, select Kernel → Restart) and then **run all cells** (in the menubar, select Cell → Run All).

Make sure you fill in any place that says YOUR CODE HERE or "YOUR ANSWER HERE", as well as your name and collaborators below:

```
NAME = "Jooho Kim"
COLLABORATORS = ""
```

```
## import libraries needed
## make sure you run this code first!
```

```
library(testthat)
library(digest)
library(stringr)
```

✓ Question 1

Create a function called `convTemp`, which converts temperature from one of the units - Fahrenheit, Celsius or Kelvin, to another from this list. Your function should have 3 input parameters:

- `x` - which is a vector of (real) numbers
- `from` - which is a character parameter, having possible values "C" (default), "F", or "K"
- `to` - which is a character parameter, having possible values "F" (default), "C", or "K"

Recall that if a temperature has value F in Fahrenheits, C in Celsius and K in Kelvin, then

$$F = \frac{9}{5} \cdot C + 32$$

$$K = C + 273.15$$

If `from` and `to` happen to be the same units (i.e. the same character variables from the set { "C", "F", "K" }), then your function should return the same vector as the input vector, but should previously print out the `warning`, which warns a user that these two are the same (so the function is not useful in that case). Please use the following command for the warning (**do not change this command, not even a single character!**)

```
warning("Your 'from' parameter is the same as your 'to' parameter!")
```

Here are some examples of what your function should return.

```
convTemp(c(-10,0,10))
```

```
14 32 50
```

```
# YOUR CODE HERE
```

```
convTemp <- function(x, from="C", to="F") {
```

```
  # Edge case: if 'from' and 'to' are the same
```

```
  if (from == to) {
```

```
    warning("Your 'from' parameter is the same as your 'to' parameter!")
```

```
    return (x)
```

```
  }
```

```
  # Convert input to Celsius
```

```
  # If 'from' is already "C", keep as is
```

```
  if (from == "F") {
```

```
    x <- (x - 32) * 5/9
```

```
  } else if (from == "K") {
```

```
    x <- x - 273.15
```

```
  }
```

```
  # Convert from Celsius to target unit
```

```
  # If 'to' is "C", keep as is
```

```
  if (to == "F") {
```

```
    x <- x * 9/5 + 32
```

```
  } else if (to == "K") {
```

```
    x <- x + 273.15
```

```
  }
```

```
  return (x)
```

```
}
```

```
convTemp(c(35,40,45), from="F", to="K")
```

```
➦ 274.816666666667 · 277.594444444444 · 280.372222222222
```

```
## Check conversion from default C to default F
```

```
if (test_that(desc="default Celsius to default Fahrenheit", code={
```

```
  expect_equal(convTemp(c(20,30)), c(68,86))
```

```
}) != TRUE) stop("Sorry, wrong conversion from default C to default F!")
```

```
➦ Test passed 🌈
```

```
## check conversion from F to C
```

```
if (test_that(desc="from Fahrenheit to Celsius", code={
```

```
expect_equal(convTemp(c(32,50,104),from="F", to="C"), c(0,10,40))
}) != TRUE) stop("Sorry wrong conversion from F to C")
```

⇒ Test passed 🎉

```
## check conversion from default C to K, and from F to K (to 5 decimal places)
```

```
if (test_that(desc="from default C to K",code={
  expect_equal(norm(convTemp(c(16,30),to="K") - c(289.15,303.15),type="2") < 1e-5, T)
}) != TRUE) stop("Sorry, wrong conversion from C to K!")
```

```
if (test_that(desc="", code={
  expect_equal(norm(
    convTemp(c(35,40,45), from="F", to="K") - c(274.816666667,277.59444444,280.37222222),type="2") < 1e-5, TRUE)
}) != TRUE) stop("Sorry, wrong conversion from F to K")
```

⇒ Test passed 🎉
Test passed 🐱

```
## check some conversions (hidden tests)
```

```
## check some conversions (hidden tests)
```

```
## check some conversions (hidden tests)
```

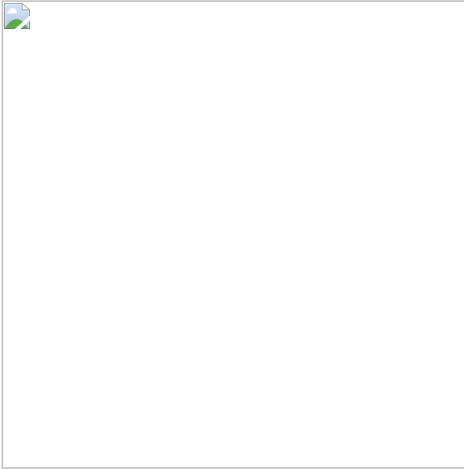
```
# install.packages("jpeg")
# library(jpeg)
# img <- readJPEG("/content/hw2pic1.jpg")
# plot(1:2, type='n')
# rasterImage(img, 1, 1, 2, 2)
```

✓ Question 2

Create an R function `pwfun()`, which computes values of the piece-wise (mathematical) function `pwfun` defined in the following way:

$$\text{pwfun}(x) = \begin{cases} -2x - 2 & , \quad x < -1 \\ 0 & , \quad -1 \leq x \leq 1 \\ x^2 - 1 & , \quad 1 < x \end{cases}$$

Its graph is as follows.



Your R function `pwfun()` should also accept `x` as a vector of arbitrary length and return vector of the same length with entries defined as values of the mathematical function `pwfun`, evaluated at the corresponding entries of vector `x`.

Here are some input vectors and what you should get as a corresponding output.

```
pwfun(-1)
```

```
0
```

```
pwfun(c(-2, 0.5, 3))
```

```
2 0 8
```

```
x = c(-3, -2, -1, 0, 1, 2, 3.2)
```

```
pwfun(x)
```

```
4 2 0 0 0 3 9.24
```

Hint:

- Obviously, one way to solve this is using `if-elseif-else` statment. Apart from that, you can also take a look at the R function `ifelse()`, which you could call twice.
- Be careful: because R has assignment operator `<-`, code lines `a < -1` and `a <-1` do **NOT** have the same meaning! While `a < -1` checks whether variable `a` is less than -1, `a <-1` means the same as `a <- 1`, which is assigning the value 1 to the variable `a`.

```

# YOUR CODE HERE
pwfun <- function(x) {
  return (ifelse (x < -1, -2 * x -2, ifelse(x <= 1, 0, x^2 - 1)))
}

# check whether pwfun(x) is a vector of appropriate length

if (test_that(desc="", code={
  expect_equal(length(pwfun(-3)), 1)
}) != TRUE) stop("Sorry, pwfun(-3) should be a vector of length 1")

if (test_that(desc="", code={
  expect_equal(length(pwfun(-3)), 1)
}) != TRUE) stop("Sorry, pwfun(c(-3,6)) should be a vector of length 1")

if (test_that(desc="", code={
  expect_equal(length(pwfun(c(-3,1,-6,12,4))), 5)
}) != TRUE) stop("Sorry, pwfun(c(-3,1,-6,12,4)) should be a vector of length 5")

🔗 Test passed 🤔
Test passed 🌈
Test passed 😊

# check whether pwfun(x) is correct for several vectors x

if (test_that(desc="", code={
  expect_equal(pwfun(-1), 0)
}) != TRUE) stop("Sorry, pwfun(-1) should be 0")

if (test_that(desc="", code={
  expect_equal(pwfun(7),
              48)
}) != TRUE) stop("Sorry, pwfun(7) should be 48")

if (test_that(desc="", code={
  expect_equal(pwfun(c(-2, 0.5, 3)), c(2,0,8))
}) != TRUE) stop("Sorry, wrong answer")

if (test_that(desc="", code={
  expect_equal(pwfun(c(-3,-2,-1,0,1,2,3.2)), c(4, 2, 0, 0, 0, 3, 9.24))
}) != TRUE) stop("Sorry, pwfun(c(-3,-2,-1,0,1,2,3.2)) should be c(4, 2, 0, 0, 0, 3, 9.24)")

```

↔ Test passed 🏆
↔ Test passed 🏆
↔ Test passed 🏆
↔ Test passed 🏆

```
# check whether pwfun(x) is correct for some values x (hidden test)
```

```
# check whether pwfun(x) is correct for some more values x (hidden test)
```

```
# more hidden testing about pwfun(x)
```